

RECON ROVER V2 - TECHNICAL DESIGN MANUAL

Version: 1.0.0 Architecture Version: 8.9 Document Version: 1.0 Project Repository: Recon Rover V2

Revision History

Date	Version	Description	Author
2026-07-16	1.0	Initial Release of Technical Design Manual	AI Engineering Team

1. Executive Summary

This document serves as the definitive Software Design Description (SDD) for Recon Rover V2. It explains how the system is designed, why it was designed that way, and how every subsystem interacts, based strictly on the Phase 1.0 through 8.9 implemented repository.

2. Architectural Design Principles

The Recon Rover V2 is built upon three non-negotiable engineering principles:

- Total Hardware/Cognitive Decoupling:** The ESP32 handles all hard-real-time PWM and interrupt constraints. The Raspberry Pi handles asynchronous AI orchestration.
- EventBus Isolation:** Subsystems (Vision, Speech, LLM, etc.) do NOT invoke each other. They communicate strictly via an in-memory Pub/Sub EventBus.
- Graceful Degradation:** The AI layers can crash, throttle, or hang without affecting the physical safety limits hardcoded on the ESP32.

3. Subsystem Breakdown

3.1 Demo Runtime (Mission Executive)

- Purpose:** Macro-orchestrator of autonomous missions.
- Responsibilities:** Evaluates readiness, loads scenarios, dispatches sequence events.
- Folder:** `core/ai/demo/`
- Modules:** `demo_runtime.py`, `scenario_manager.py`, `integration_coordinator.py`
- Classes:** `DemoRuntime`, `IntegrationCoordinator`, `SystemReadiness`
- Configuration:** Derived from `.env` scenario files.
- Dependencies:** `EventBus`.
- Events Published:** `demo.start`, `demo.ready`, `demo.shutdown`
- Events Consumed:** `health.*`, `agent.consensus`
- Thread Safety:** Fully Single-Threaded AsyncIO.
- Async Behavior:** Non-blocking state machine loops.

- **Failure Modes:** Fails if `SystemReadiness` timeouts (e.g., missing hardware).
- **Recovery:** Emits `demo.recovery` and pauses scenario execution.

3.2 Vision Subsystem

- **Purpose:** Environment perception and object detection.
- **Responsibilities:** Capturing frames, executing YOLO inferences, semantic tagging.
- **Folder:** `core/ai/runtime/vision/`
- **Modules:** `vision_runtime.py`, `object_detector.py`, `camera_provider.py`
- **Configuration:** `CAMERA_FPS`, `VISION_MODEL_PATH`
- **Events Published:** `vision.detection`, `vision.fps`
- **Events Consumed:** `vision.cmd.capture`
- **Performance:** Aims for 10-15 FPS on RPi5 CPU; 30+ with Coral Edge TPU (if present).
- **Internal Pipeline:** USB Frame -> Buffer -> YOLO ONNX -> Bounding Box Extractor -> EventBus.

3.3 Speech Subsystem

- **Purpose:** Natural language auditory processing.
- **Responsibilities:** Wake word detection, Speech-to-Text (Whisper), Text-to-Speech (Piper).
- **Folder:** `core/ai/runtime/speech/`
- **Modules:** `speech_runtime.py`, `speech_recognizer.py`, `wake_word_detector.py`
- **Events Published:** `speech.recognized`, `speech.audio_out`
- **Events Consumed:** `speech.cmd.say`

3.4 LLM Subsystem

- **Purpose:** Cognitive engine for reasoning and NLP.
- **Responsibilities:** Provider-agnostic inference generation.
- **Folder:** `core/ai/runtime/llm/`
- **Modules:** `llm_runtime.py`, `provider_manager.py`, `ollama_provider.py`
- **Events Published:** `llm.response`
- **Events Consumed:** `llm.request`
- **Optimization:** Streams tokens directly to EventBus to reduce TTFT (Time To First Token).

3.5 RAG Subsystem

- **Purpose:** Contextual memory storage.
- **Responsibilities:** Vectorizing mission history and static manuals via ChromaDB.
- **Folder:** `core/ai/runtime/rag/`
- **Modules:** `rag_runtime.py`, `vector_database.py`, `semantic_search.py`
- **Events Published:** `rag.results`
- **Events Consumed:** `rag.query`, `rag.index`

3.6 Tool Execution Subsystem

- **Purpose:** Safe sandbox for LLM-driven hardware interaction.
- **Responsibilities:** Parsing LLM JSON, validating permissions, executing Python functions.

- **Folder:** `core/ai/runtime/tools/`
- **Modules:** `tool_runtime.py, tool_validator.py, tool_executor.py`
- **Events Published:** `tool.result, navigation.cmd` (via `NavigationTool`)
- **Events Consumed:** `tool.invoke`

3.7 Multi-Agent Subsystem

- **Purpose:** Complex task subdivision and reasoning consensus.
- **Responsibilities:** Managing sub-agents (Planner, Vision, Memory) over a shared blackboard.
- **Folder:** `core/ai/runtime/agents/`
- **Modules:** `agent_runtime.py, blackboard_runtime.py, consensus_manager.py`
- **Events Published:** `agent.consensus, agent.task_assigned`
- **Events Consumed:** `agent.msg`

3.8 Optimization Subsystem

- **Purpose:** Hardware protection and dynamic performance scaling.
- **Responsibilities:** Thermal throttling, dynamic thread pool allocation.
- **Folder:** `core/ai/runtime/optimization/`
- **Modules:** `optimization_runtime.py, thermal_manager.py, thread_pool_manager.py`
- **Events Published:** `optimization.throttle`
- **Events Consumed:** `benchmark.telemetry` (for load assessment)

3.9 Benchmark Subsystem

- **Purpose:** Passive, zero-overhead telemetry observation.
 - **Responsibilities:** Aggregating CPU, RAM, Latency, and Throughput stats.
 - **Folder:** `core/ai/runtime/benchmark/`
 - **Modules:** `benchmark_runtime.py, latency_profiler.py, metrics_database.py`
 - **Events Published:** `benchmark.telemetry`
 - **Events Consumed:** All `*.*` (Passive observer)
-

4. Phase Implementation Summary

- **Phase 8.0 (Vision):** Implemented ONNX-based object detection. Verified 15FPS processing.
- **Phase 8.1 (Speech):** Implemented Whisper ASR and Piper TTS. Verified 95% wake word accuracy.
- **Phase 8.2 (LLM):** Implemented agnostic provider interface. Standardized on Ollama backend.
- **Phase 8.3 (RAG):** Implemented ChromaDB local vector store.
- **Phase 8.4 & 8.5 (Tools):** Implemented secure LLM JSON parser and execution sandbox.
- **Phase 8.6 (Multi-Agent):** Implemented Blackboard and Supervisor architectures for consensus.
- **Phase 8.7 (Optimization):** Implemented thermal throttling algorithms.
- **Phase 8.8 (Benchmark):** Implemented SQLite telemetry persistence.

- **Phase 8.9 (Demo):** Implemented overarching scenario state machine for autonomous execution.

(End of Technical Design Manual)